

Evaluating Peer & Piece Selection Algorithms in the BitTorrent Protocol

Lukiana Weger

Student

mail@weger.dev

Abstract—I present an empirical study on the performance of custom piece and peer selection heuristics in BitTorrent swarms using a custom Rust client.

Index Terms—BitTorrent, Peer-to-Peer, Algorithms, Network Performance

I. INTRODUCTION

In peer-to-peer networks, piece selection directly impacts performance as described in [1, p. 3, 2.4], the optimistic unchoking slot...

CONTENTS

I)	Introduction	1
II)	Quick Overview of the BitTorrent Protocol	2
	II.A) The difference between DHT and Tracker ...	2
	II.B) .torrent files and magnet links	2
	II.C) info_hash	2
	II.D) Clients and Pieces	2
III)	Piece Selection	3
	III.A) Rarest-First	3
	III.A.a) Entropymaxxing	3
	III.B) Most common first	3
	III.C) Random	3
	III.C.a) Random walk with momentum	3
	III.D) Sequential	3
	III.D.a) Reverse sequential	3
	III.E) Hybrid rarest-first + sequential	3
	III.F) Availability-weighted random	3
IV)	Peer Selection	3
	IV.A) Tit-for-Tat	3
	IV.B) Anti-tit-for-tat / Altruism maximization	3
	IV.C) Optimistic	3
	IV.D) Latency-aware selection	3
	IV.E) BAR (Bounded Altruism with Reciprocity) ...	3
	IV.F) Gradient descent on peer utility	3
	IV.G) Random	3
	IV.G.a) Unchoke by peer-id lexicographic order	3
V)	Block & Pipelining Parameters	4
	V.A) In-flight Queue Depth	4
	V.B) Block Request Size	4
	V.C) Endgame Mode Trigger Threshold	4
VI)	Peer Dynamics & Choking Parameters	4
	VI.A) Unchoke Round Frequency	4
	VI.B) Unchoke Slot Count	4
	VI.C) Anti-Snubbing Sensitivity	4
VII)	Transport & Infrastructure	4
	VII.A) Async Verification Pipeline	4
	References	4

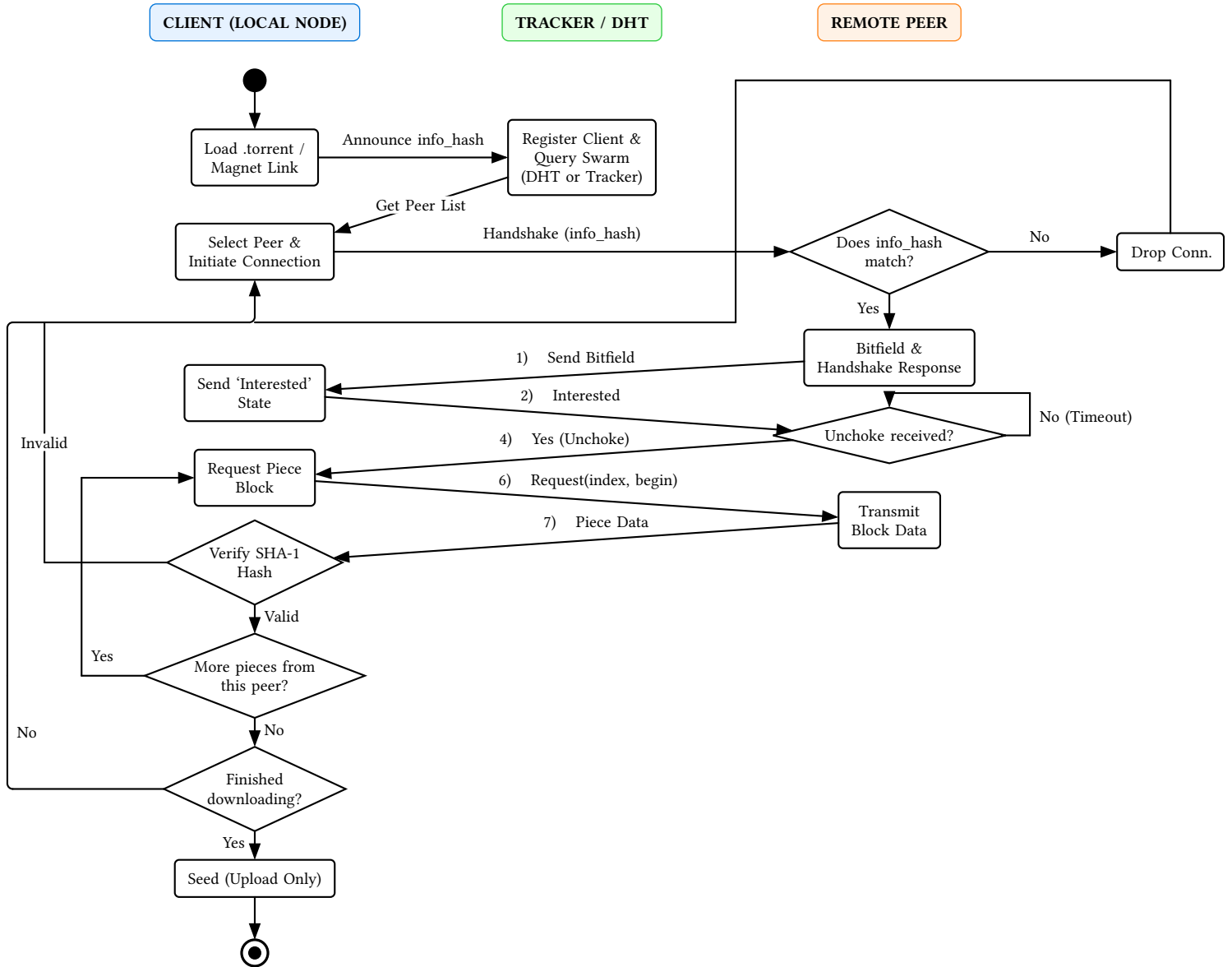


Fig. 1. Hybrid UML Activity & Sequence diagram of the BitTorrent protocol. Swimlanes separate Client, Tracker, and Peer domains, highlighting network message exchanges and retry control loops.

II. QUICK OVERVIEW OF THE BITTORRENT PROTOCOL

As shown in Fig. 1, control flow cycles between block transfers and peer re-selection

- A. The difference between DHT and Tracker
- B. .torrent files and magnet links
- C. info_hash
- D. Clients and Pieces

The actual peer- and piece selection algorithms used by the BitTorrent protocol are random peer selection and a 4 phase piece selection algorithm as mentioned in [1, p. 3, 2.4].

The peers get chosen at random because These 4 phases of piece selection are:

- 1) Strict priority: Once a block of a piece is received all the other blocks will be downloaded first before

another piece is chosen. My Implementation will do the same thing since i dont see why diverging from that practice could ever increase performance or swarm health.

- 2) The Client begins by selecting random pieces to download, until at least one piece is assembled.
- 3) It then switches to rarest first. Each peer only sees piece rareness within its own peer set not the global swarm.
- 4) Towards the end we switch into something called "Endgame mode". Once all the pieces are requested but the client is missing some block, it sends out requests for all the missing blocks to all *all* connected peers.

III. PIECE SELECTION

A. Rarest-First

a) *Entropymaxxing*: At each step pick whichever piece maximizes Shannon entropy of your local bitfield. Computationally absurd, probably converges to something close to rarest-first anyway, but the journey is the point.

Theorem: Shannon entropy

Shannon entropy in its general form measures the average unpredictability of a probability distribution

$$H(X) = - \sum_{x \in X} p(x) \cdot \log p(x) \quad (1)$$

For a binary string of length N as we have in our bitfield the naive application is just treating it as a Bernoulli distribution $\rightarrow$ what fraction of bits are ones (p_1), and which are zeros (p_0).

$$p_1 = \frac{\text{number of pieces you have}}{N} \quad (2)$$

$$p_0 = 1 - p_1 \quad (3)$$

which gives us

$$H_{\text{naive}} = -(p_1 \cdot \log(p_1) + p_0 \cdot \log(p_0)) \quad (4)$$

Our naive approach will peak at $p_{1/2} 0.5$ and will have zero at both extreme points. This is almost useless for selecting new pieces because it just tells us about the current entropy of the bitfield and not *which* pieces you have. (101010... & 111000... will have the same entropy).

What will actually give you a meaningful value is the joint distribution of local and swarm bitfield. for each piece i define, $p(i)$ = fraction of peers in the swarm that have piece i . Using this technique your local bitmap acts like a mask revealing the pieces that arent present in the local bitmask(S), among those you want to pick the piece that maximizes the amount of new information you gain (of course relative to what the swarm can provide). A reasonable objective would be to take the piece with the highest surprise value out of a set defined as:

$$I = \{-\log_2(p_i) \mid i \in S\} \quad (5)$$

This will literally just result in being rarest-first reframed in information-theoretic language, which is a neat result because it proves that the rarest-first approach actually implicitly maximizes entropy.

Another interpretation of maximizing entropy could be maximizing local entropy by defining some criteria by which the bit field maximizes entropy, this for me is a arbitrary criteria like “most spread out” and has no scientific value. If we follow this thought: you just use something like run-length-entropy (RLE) by which a bitfield like 11110000 has low run-length entropy (two long runs). A bitfield like 10100101 has higher run-length entropy (many short runs). Maximizing this pushes you toward scattered, interleaved piece acquisition. $\rightarrow$ practically this pushes you towards something like random piece selection with a sort of repulsion effect.

“Entropymaxxing” is theoretically elegant and practically reduces down to rarest-first or pretty much just randomized selection depending on the definition. The computing overhead added by calculating the Shannon Entropy of any bitfield doesn't make this any viable competitor what so ever.

B. Most common first

literally the inverse of rarest-first. Greedily download what everyone has. Swarm health collapses hilariously fast; good stress test for how badly broken a strategy can get.

C. Random

a) *Random walk with momentum*: pick a random piece, then with 80% probability pick an adjacent piece next, else re-randomize. Creates weird locality clusters in the bitfield

D. Sequential

a) *Reverse sequential*:

E. Hybrid rarest-first + sequential

what qBittorrent does for streaming) – rarest-first globally but sequential within a sliding window. You currently treat these as mutually exclusive.

F. Availability-weighted random

weighted random where weights are inversely proportional to piece availability. Middle ground between pure random and strict rarest-first; avoids the “everyone rushes the rarest” thundering herd.

IV. PEER SELECTION

A. Tit-for-Tat

B. Anti-tit-for-tat / Altruism maximization

preferentially unchoke peers who upload the least to you. Will get you exploited by every free-rider in the swarm; interesting to see how fast

C. Optimistic

D. Latency-aware selection

preferring peers with lower RTT vs. purely tit-for-tat. Does proximity win over reciprocation? Especially relevant in sparse swarms.

E. BAR (Bounded Altruism with Reciprocity)

game-theoretic model that tolerates some free-riding up to a bound. Interesting to benchmark against strict tit-for-tat in mixed honest/freeloading swarms.

F. Gradient descent on peer utility

model each peer as having a utility score (upload rate $\times$ availability $\times$ latency⁻¹), gradient-descent the unchoke allocation every round. Probably overkill but defensible.

G. Random

a) *Unchoke by peer-id lexicographic order*: since peer_id are often semi-random in the first place this should be roughly equivalent to random with extra steps.

V. BLOCK & PIPELINING PARAMETERS

A. *In-flight Queue Depth*

BitTorrent requests pieces in smaller 16 KiB blocks. How many block requests are kept in-flight per peer (e.g., 16, 128, 250)? Lower values underutilize bandwidth; higher values increase latency or cause buffer bloat.

B. *Block Request Size*

Standard is 16 KiB, but what happens to TCP/ μ TP overhead and throughput if you test 8 KiB, 32 KiB, or 64 KiB blocks?

C. *Endgame Mode Trigger Threshold*

When a download is nearly finished (e.g., last 20 blocks or last 1–2%), clients send duplicate requests to all peers to finish quickly. Tuning when and how aggressively to enter endgame mode affects network overhead vs. completion tail-latency.

VI. PEER DYNAMICS & CHOKING PARAMETERS

A. *Unchoke Round Frequency*

Standard BitTorrent re-evaluates top downloading peers every 10 seconds and rotates optimistic unchokes every 30 seconds. What happens if you run choking rounds every 2s vs 15s?

B. *Unchoke Slot Count*

Fixed number of active upload slots (e.g., 4 or 8 slots) versus dynamic bandwidth-based allocation per peer.

C. *Anti-Snubbing Sensitivity*

If a peer doesn't send a block for X seconds (standard ~60s), mark them as "snubbed" and stop uploading to them.

VII. TRANSPORT & INFRASTRUCTURE

A. *Async Verification Pipeline*

Does offloading SHA-1/SHA-256 piece hash checks to a background worker pool impact disk I/O bottlenecks compared to inline verification?

REFERENCES

- [1] B. Cohen, "Incentives Build Robustness in BitTorrent," *Workshop on Economics of Peer-to-Peer Systems*, vol. 6, pp. 35–40, 2003.
- [2] L. Qiu, S. R. Seshadri, and R. H. Katz, "Analysis of a BitTorrent-like Performance Model," in *ACM SIGCOMM Computer Communication Review*, 2004, pp. 367–378.